



République Tunisienne
Ministère de l'Enseignement Supérieur

ISSET Tataouine
Département Technologies de l'Informatique

Rapport de stage de perfectionnement DSI (Technicien)

Réalisé par :
Mohamed Karrab

Encadré par :
Mr Anouar Karrab

Année universitaire : 2025 – 2026

Remerciements

Au terme de mon stage, je profite de l'occasion pour exprimer mes vifs remerciements au professeur SGHAIER NAJJARI, directeur générale de l'institut des régions arides de Médenine (IRA) qui m'a permis de réaliser ce stage au sein de son établissement. Il est également de mon devoir de remercier vivant, monsieur Oussar Mohamed, directeur du service de formation des régions Arides a l'IRA, pour m'avoir accueilli. Je tiens à remercier mon encadreur, Monsieur KARRAB LANOUAR, pour son soutien professionnel et ses conseils mais aussi pour m'avoir permis de travailler dans très bonnes conditions en mettant à ma disposition le matériel logistique dont j'avais besoin.

Table des matières

Remerciements	1
Introduction générale	6
1 Présentation du cadre du stage	7
1.1 Présentation de la société	7
1.2 Présentation de l’Institut des Régions Arides (IRA)	7
1.2.1 Présentation globale	7
1.2.2 Organigramme de l’IRA	9
1.3 Problématique	10
1.4 Solution proposée	10
1.5 Conclusion	10
2 Spécification des besoins	11
2.1 Introduction	11
2.2 Besoins fonctionnels	11
2.2.1 Identification des acteurs	11
2.2.2 Identification des besoins	11
2.2.3 Modélisation des besoins	12
2.3 Besoins non fonctionnels	16
2.4 Conclusion	16
3 Étude conceptuelle	17
3.1 Introduction	17
3.2 Vue statique du système	17
3.2.1 Modèle conceptuel de données (MCD)	18
3.2.2 Dictionnaire de données	18
3.2.3 Modèle relationnel de données (MRD)	18
3.3 Vue dynamique du système	19
3.4 Déploiement du système	19
3.5 Conclusion	20
4 Réalisation	21
4.1 Introduction	21
4.2 Environnement de travail	21
4.2.1 Environnement matériel	21
4.2.2 Environnement logiciel	21
4.3 Architecture de l’application	22
4.4 Présentation des interfaces	22
4.5 Intégration de la base de données	26

4.6	Intégration de la partie IA (Chatbot)	27
4.7	Conclusion	27
Conclusion générale		28
Annexes		30
4.8	Annexe A : Code source principal	30
4.9	Annexe B : Schémas et diagrammes détaillés	30
4.10	Annexe C : Fichiers de configuration	30
4.11	Annexe D : Tests et validation	31

Table des figures

1.1	Organigramme de l'Institut des Régions Arides	9
2.1	Diagramme des cas d'utilisation généralisé de l'application	12
2.2	Diagramme des cas d'utilisation 1	13
2.3	Diagramme des cas d'utilisation 2	14
2.4	Diagramme des cas d'utilisation 3	14
2.5	Diagramme des cas d'utilisation 4	15
2.6	Diagramme des cas d'utilisation 5	15
3.1	Diagramme de classe de l'application	18
4.1	Page de connexion	23
4.2	Page de inscription	23
4.3	Page de Tableau de bord	23
4.4	Page de données	24
4.5	Page de indices	24
4.6	Page de provinces	25
4.7	Chatbot	25
4.8	Carte interactive	26
4.9	Upload de fichiers	26

Liste des tableaux

2.1	Tableau global des cas d'utilisation	13
3.1	Dictionnaire des données principales	19

Introduction générale

Dans un monde où les données géographiques deviennent de plus en plus essentielles pour la prise de décision, la gestion et l'analyse des informations spatiales représentent un défi majeur. Ce projet, intitulé "Plateforme Geo API", vise à développer une application web complète permettant de centraliser, gérer et visualiser des données géographiques relatives aux provinces et aux indices de données environnementales.

L'objectif principal de cette plateforme est de fournir une interface unifiée pour les utilisateurs et administrateurs, facilitant l'accès aux données géographiques, la gestion des indices climatiques et environnementaux, ainsi que l'interaction via un chatbot intelligent. Développée avec une architecture moderne séparant le backend (Spring Boot) et le frontend (Angular), cette application intègre des fonctionnalités avancées telles que l'authentification sécurisée, la gestion des fichiers CSV, et l'intégration d'une base de données relationnelle.

Ce rapport présente l'ensemble du processus de développement, depuis la présentation du projet jusqu'à sa réalisation complète. Il est structuré en quatre chapitres : la présentation du projet, la spécification des besoins, l'étude conceptuelle et la réalisation. Chaque chapitre détaille les étapes clés du développement, les choix technologiques et les défis rencontrés.

Chapitre 1

Présentation du cadre du stage

1.1 Présentation de la société

Avant l'élaboration de tout projet, il est indispensable de prendre connaissance de la situation actuelle de l'organisme d'accueil ainsi que des technologies existantes. Dans ce chapitre, nous commencerons par présenter l'organisme dans lequel s'est déroulé le stage. Ensuite, nous donnerons un aperçu général de ses structures et missions. Enfin, nous exposerons le contexte général du projet réalisé.

1.2 Présentation de l'Institut des Régions Arides (IRA)

1.2.1 Présentation globale

Attributions

Créé en vertu de la loi n° 76-6 du janvier, l'Institut des Régions Arides (IRA) de Médenine a pour principales missions :

- La réalisation des recherches nécessaires au développement du secteur agricole, à la protection et à la conservation des ressources naturelles, ainsi qu'à la lutte contre la désertification dans les régions arides et désertiques.
- La formation et les stages de perfectionnement des techniciens et chercheurs spécialisés en aridologie et en lutte contre la désertification.
- La participation à la vulgarisation et à l'encadrement technique du secteur agricole en vue de la conservation des ressources naturelles et de la protection contre le risque de désertification.
- La réalisation d'études de projets de développement intégré à l'échelle nationale, sous-régionale et internationale.
- L'animation et la coordination des activités et travaux des différents organismes intervenant dans le domaine de la lutte contre la désertification et de la protection des ressources naturelles.

Structures scientifiques et techniques

Pour accomplir ses missions, l'IRA dispose de plusieurs structures scientifiques et techniques, notamment :

- Laboratoire d'Erémologie et de lutte contre la désertification.
- Laboratoire des parcours et plantes spontanées des zones arides et désertiques.

- Laboratoire d'élevage et de faune sauvage.
- Laboratoire d'économie et sociétés rurales.
- Unité de vulgarisation et des projets de développement.
- Unité d'information et de documentation des régions arides.
- Unités de formation.

Recherche érémologique et lutte contre la désertification

Les principales activités de cette unité sont :

- La surveillance et l'observation globales de la désertification.
- L'étude de l'érosion éolienne et la lutte contre l'ensablement.
- La valorisation des eaux de ruissellement et des ressources en sol dans les zones arides et désertiques.

Parcours et plantes spontanées des zones arides et désertiques

Cette unité s'intéresse principalement à :

- La conservation et l'observation des plantes spontanées des zones arides et désertiques.
- L'amélioration et la gestion des parcours dans les zones arides.
- L'amélioration génétique des plantes pastorales.

Élevage et faune sauvage en zones arides

Les travaux de cette unité portent sur :

- La réhabilitation de l'élevage camelin et l'amélioration de sa productivité.
- L'amélioration des performances de production des caprins dans les milieux arides et oasiens.
- La conservation et la valorisation de la faune sauvage dans les zones arides et désertiques.

Aridoculture oasienne

Cette unité a pour objectifs :

- La conservation et la valorisation des ressources phylogénétiques.
- L'amélioration de la conduite des cultures en zones arides.
- La formation des techniciens supérieurs.

Afin de pallier le manque de spécialistes en aridoculture et en gestion durable des ressources naturelles, l'IRA développe plusieurs activités de formation telles que :

- Les stages de perfectionnement au profit des techniciens.
- L'encadrement des étudiants stagiaires (cycles académique et postuniversitaire).
- L'organisation de cours internationaux postuniversitaires.

Stations régionales et zones d'intervention

Le siège de l'Institut des Régions Arides est situé à El Fjé, à 22,5 km de Médenine sur la route Médenine–Djerba. L'IRA dispose de quatre stations expérimentales réparties sur l'ensemble des gouvernorats du sud tunisien.

Coopération nationale et internationale

Les programmes de recherche et la valorisation de leurs résultats sont réalisés en étroite collaboration avec plusieurs partenaires nationaux et internationaux, notamment :

- Les universités et établissements de recherche tunisiens.
- Les organismes de développement.
- Les établissements d'enseignement.
- Les agriculteurs, éleveurs et autres acteurs du secteur.

Vulgarisation

Les actions de vulgarisation de l'IRA de Médenine s'adressent principalement aux agriculteurs, éleveurs, enseignants ainsi qu'aux élèves des disciplines des sciences naturelles et de la géographie.

Centre d'information, informatique, documentation et archives (DIIDA)

Le centre d'information et de documentation des régions arides joue un rôle essentiel dans le développement des applications scientifiques, la gestion moderne des applications techniques et administratives ainsi que la coordination des activités informatiques. Il se compose de :

- L'unité de publication.
- L'unité de documentation.
- L'unité de statistiques et informatique.

1.2.2 Organigramme de l'IRA

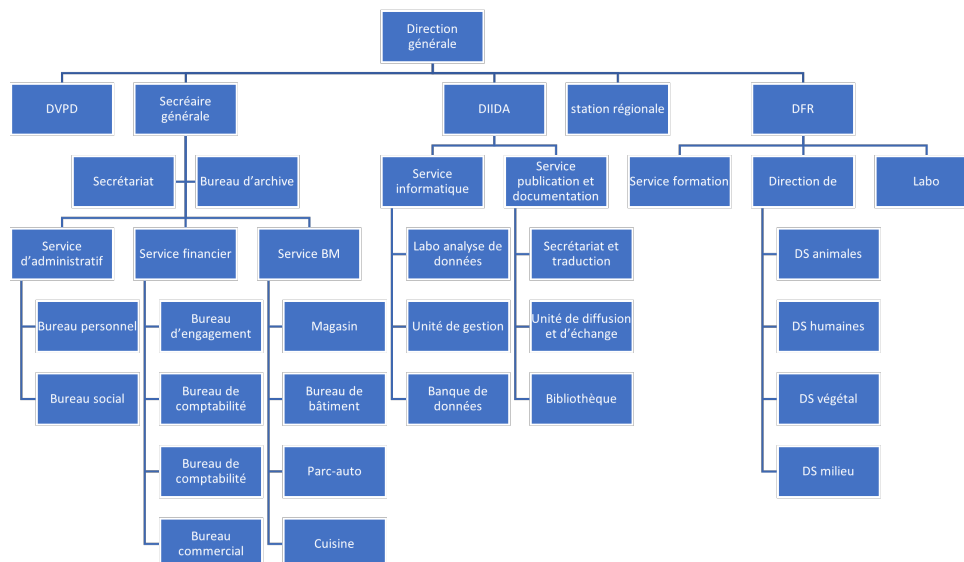


FIGURE 1.1 – Organigramme de l'Institut des Régions Arides

1.3 Problématique

La problématique principale identifiée est l'absence d'une plateforme unifiée et accessible pour la gestion et la visualisation des données géographiques. Les défis incluent : - La dispersion des sources de données, rendant difficile la consolidation des informations. - Le manque d'outils d'analyse spatiale accessibles aux non-experts. - L'absence d'interfaces interactives pour la visualisation des données géographiques. - La nécessité d'intégrer des fonctionnalités d'intelligence artificielle pour améliorer l'expérience utilisateur. Cette situation limite l'exploitation optimale des données géographiques pour la prise de décision et la recherche.

1.4 Solution proposée

La solution proposée est une plateforme web moderne composée de deux parties principales : - Un backend REST API développé avec Spring Boot, gérant la logique métier, l'authentification sécurisée, et l'intégration avec une base de données relationnelle (H2/PostgreSQL). - Un frontend développé avec Angular, offrant une interface utilisateur réactive et intuitive pour la visualisation des données sur des cartes interactives. La plateforme intègre également un chatbot basé sur l'intelligence artificielle pour assister les utilisateurs dans leurs requêtes, ainsi qu'un système de gestion des fichiers CSV pour l'importation des données. Cette architecture assure une séparation claire des responsabilités, une scalabilité et une maintenabilité élevées.

1.5 Conclusion

Ce premier chapitre a permis de présenter le cadre du stage au sein de l'Institut des Régions Arides (IRA) de Médenine, en exposant ses missions, ses structures de recherche et ses activités de formation et de vulgarisation. Cette présentation a mis en évidence le rôle stratégique de l'IRA dans le développement agricole, la protection des ressources naturelles et la lutte contre la désertification. Elle pose ainsi les bases nécessaires pour le chapitre suivant, consacré à la spécification des besoins du projet, en lien avec les objectifs et les contraintes de l'Institut.

Chapitre 2

Spécification des besoins

2.1 Introduction

La spécification des besoins constitue une étape cruciale dans le développement de tout système informatique. Pour la Plateforme Geo API, cette phase a permis d'identifier clairement les exigences fonctionnelles et non fonctionnelles, en tenant compte des acteurs impliqués et de leurs interactions avec le système. Cette analyse s'appuie sur une compréhension approfondie du domaine géospatial et des besoins des utilisateurs finaux, qu'ils soient administrateurs ou simples utilisateurs.

2.2 Besoins fonctionnels

2.2.1 Identification des acteurs

Le système distingue deux catégories principales d'acteurs :

- **Administrateur** : utilisateur privilégié responsable de la gestion complète de la plateforme. Il dispose de tous les droits d'administration et de maintenance, lui permettant de configurer le système, gérer les utilisateurs et superviser l'intégrité des données.
- **Utilisateur** : utilisateur final qui interagit principalement avec les données et les fonctionnalités de visualisation. Il peut s'agir de chercheurs, de décideurs politiques ou de tout citoyen intéressé par l'analyse et la consultation des données géographiques.

2.2.2 Identification des besoins

Besoins de l'administrateur

L'administrateur doit disposer d'un panel complet de fonctionnalités de gestion : - Gestion des comptes utilisateurs : création, modification, suppression et attribution de rôles. - Gestion des données géographiques : ajout, modification et suppression des informations sur les provinces. - Gestion des indices de données : administration des indices environnementaux et climatiques, y compris l'importation en masse via des fichiers CSV. - Supervision du système : monitoring des performances, gestion des logs et maintenance de la base de données. - Configuration du chatbot : paramétrage des réponses et des règles d'interaction.

Besoins de l'utilisateur

L'utilisateur bénéficie d'une interface centrée sur la consultation et l'analyse : - Consultation des données : accès aux informations détaillées sur les provinces Tunisiennes (nom, coordonnées géographiques). - Visualisation cartographique : affichage interactif des données sur une carte, avec possibilité de filtrage par critères (province, type d'indice, période temporelle). - Gestion des indices : recherche et consultation des données environnementales (précipitations, températures, etc.) avec possibilité d'exportation. - Interaction avec le chatbot : pose de questions en langage naturel pour obtenir des informations ou des analyses sur les données. - Téléchargement de données : export des résultats de requêtes au format CSV ou JSON.

2.2.3 Modélisation des besoins

La modélisation des besoins s'appuie sur des diagrammes UML pour représenter les cas d'usage principaux : - Cas d'usage "Authentification" : connexion sécurisée pour tous les acteurs. - Cas d'usage "Gestion des provinces" : CRUD operations pour les administrateurs. - Cas d'usage "Consultation des données" : recherche et visualisation pour les utilisateurs. - Cas d'usage "Interaction chatbot" : dialogue assisté pour l'assistance utilisateur. - Cas d'usage "Import de données" : chargement de fichiers CSV pour l'alimentation de la base de données.

Ces cas d'usage mettent en évidence les interactions entre les acteurs et le système, facilitant la conception des interfaces et des workflows.

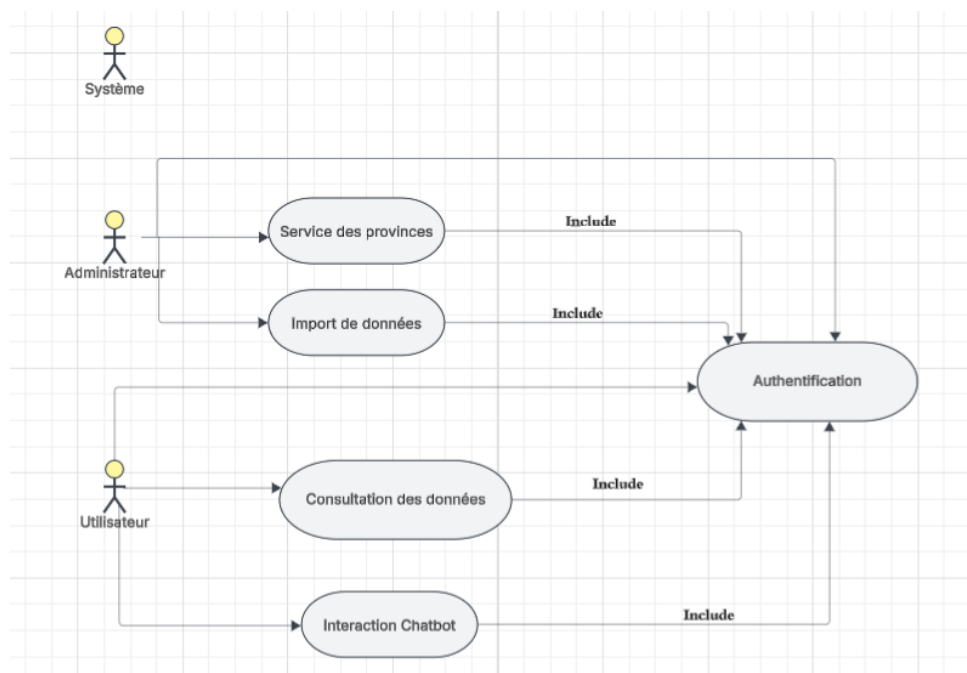


FIGURE 2.1 – Diagramme des cas d'utilisation généralisé de l'application

cas d'utilisation global

TABLE 2.1 – Tableau global des cas d'utilisation

N°	Cas d'utilisation	Acteur	Description	Préconditions	Postconditions
1	Authentification	Administrateur, Utilisateur	Accès sécurisé au système	Compte valide	Accès autorisé selon le rôle
2	Gestion des provinces	Administrateur	Gestion CRUD des provinces	Administrateur authentifié	Données mises à jour
3	Consultation des données	Utilisateur	Consultation et visualisation des données	Utilisateur authentifié	Données affichées
4	Interaction Chatbot	Utilisateur	Assistance via chatbot	Utilisateur authentifié	Réponse fournie
5	Import de données	Administrateur	Importation de fichiers CSV	Administrateur authentifié	Données enregistrées

les Diagramme des séquences :

Cas 1 : Authentification

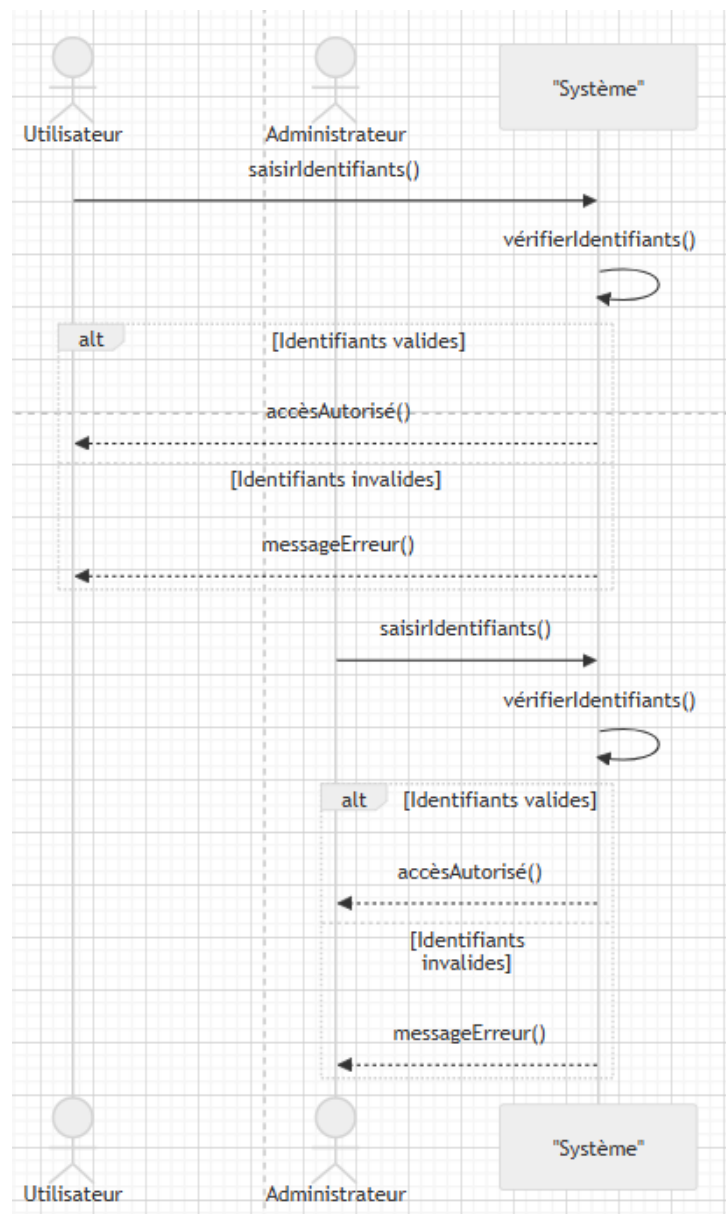


FIGURE 2.2 – Diagramme des cas d'utilisation 1

— Cas 2 : Gestion des provinces

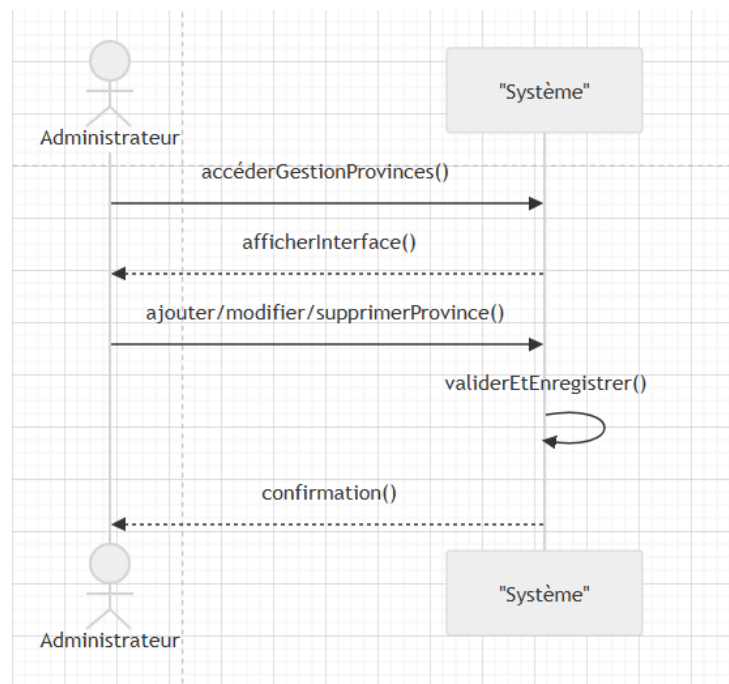


FIGURE 2.3 – Diagramme des cas d'utilisation 2

— Cas 3 : Consultation des données

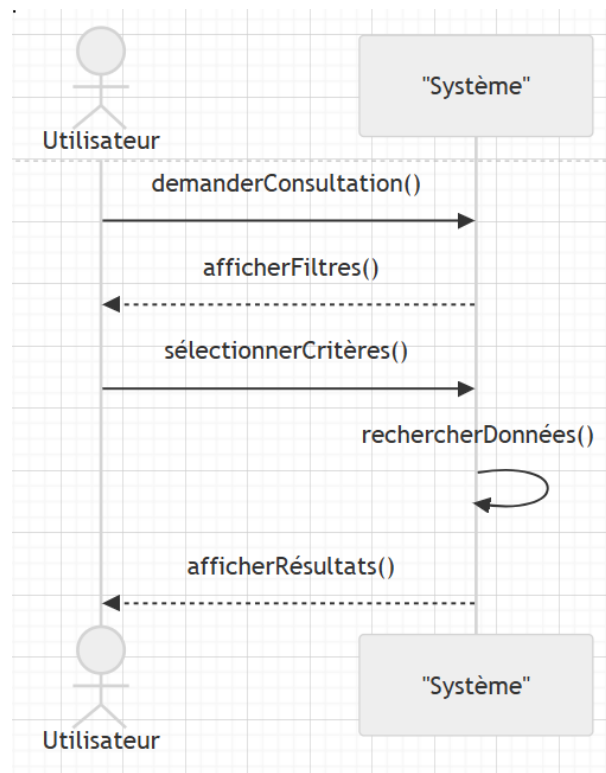


FIGURE 2.4 – Diagramme des cas d'utilisation 3

— Cas 4 : Interaction Chatbot

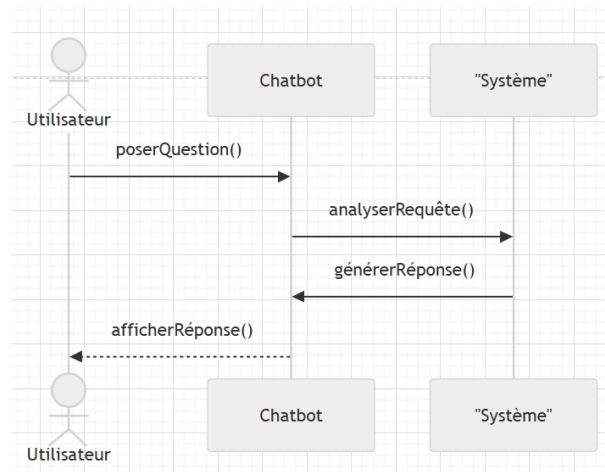


FIGURE 2.5 – Diagramme des cas d'utilisation 4

Cas 5 : Import de données

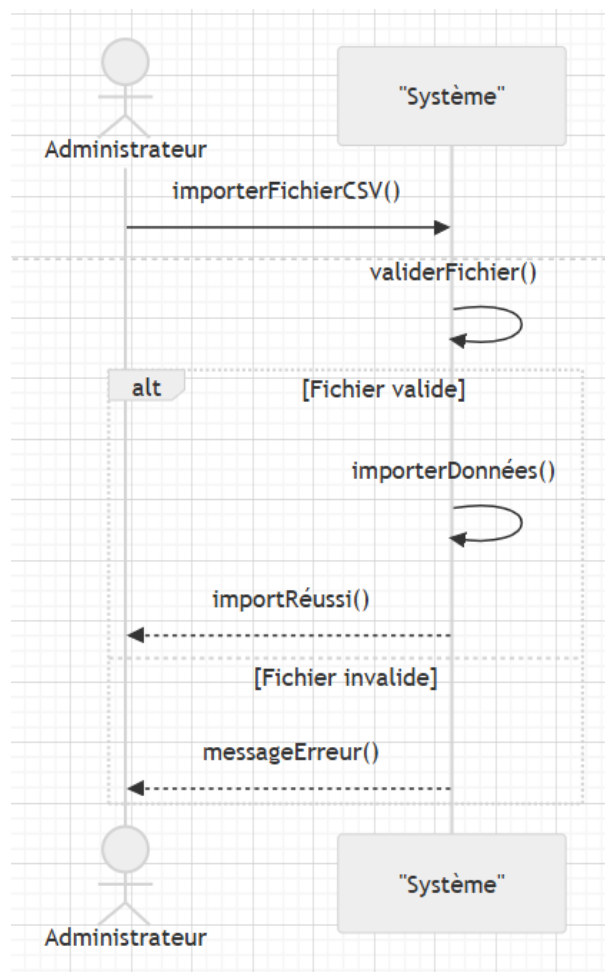


FIGURE 2.6 – Diagramme des cas d'utilisation 5

2.3 Besoins non fonctionnels

Les besoins non fonctionnels définissent les qualités attendues du système au-delà de ses fonctionnalités. Parmi eux, on distingue :

- **Sécurité** : Authentification basée sur JWT, chiffrement des données sensibles, protection contre les attaques courantes (injection SQL, XSS).
- **Performance** : Temps de réponse inférieur à 2 secondes pour les requêtes standard, capacité à gérer jusqu'à 100 utilisateurs simultanés.
- **Ergonomie** : Interface intuitive et responsive, conforme aux standards d'accessibilité web (WCAG), support multilingue (français/arabe).
- **Fiabilité** : Disponibilité de 99% du temps, mécanismes de sauvegarde automatique des données.
- **Évolutivité** : Architecture modulaire permettant l'ajout de nouvelles fonctionnalités sans refonte majeure.
- **Interopérabilité** : API REST standard pour faciliter l'intégration avec d'autres systèmes.

2.4 Conclusion

Ce chapitre a établi une spécification complète des besoins de la Plateforme Geo API, en distinguant clairement les aspects fonctionnels et non fonctionnels. Cette analyse constitue la base solide pour l'étude conceptuelle qui suivra, assurant que le système développé répondra effectivement aux attentes des utilisateurs et aux contraintes techniques.

Chapitre 3

Étude conceptuelle

3.1 Introduction

L'étude conceptuelle constitue la phase de conception architecturale et logique du système Plateforme Geo API. Cette étape vise à définir la structure statique et dynamique du système, ainsi que la conception de la base de données sous-jacente. L'approche adoptée s'appuie sur les principes de l'ingénierie logicielle moderne, privilégiant la modularité, la réutilisabilité et l'évolutivité. Cette étude s'inspire des spécifications des besoins établies précédemment et prépare le terrain pour l'implémentation technique.

3.2 Vue statique du système

La vue statique du système décrit les entités principales et leurs relations au sein de l'application. Le modèle orienté objet repose sur les classes suivantes :

- **Province** : Représente une province tunisienne avec ses attributs géographiques (nom, longitude, latitude) et sa relation avec les indices de données.
- **DataIndices** : Entité centrale contenant les données environnementales telles que l'indicateur, le pays, le code administratif, la province, le type de terre, la date, la valeur, la moyenne à long terme, l'année, le mois, la décennie, l'unité et la source.
- **Indices** : Définit les types d'indices disponibles (ex. précipitations, températures, NDVI, etc.).
- **User** : Modélise les utilisateurs du système, avec leurs informations d'authentification et leur rôle.
- **Role** : Gère les rôles utilisateurs (USER, MODERATOR, ADMIN) et les permissions associées.
- **File** : Représente les fichiers téléchargés, notamment les fichiers CSV utilisés pour l'importation des données.

Ces classes utilisent les annotations JPA pour le mapping objet-relationnel, assurant une persistance transparente des données.

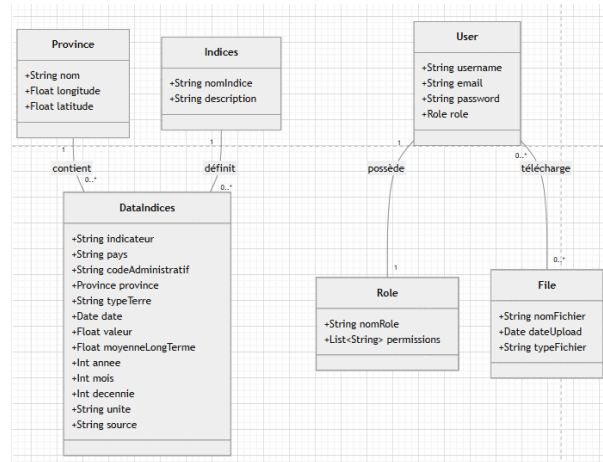


FIGURE 3.1 – Diagramme de classe de l'application

3.2.1 Modèle conceptuel de données (MCD)

Le Modèle Conceptuel de Données (MCD) représente les entités principales de l'application ainsi que leurs associations. Les relations identifiées sont les suivantes :

- **Province (1,n) – (1,1) DataIndices** : Une province peut être associée à plusieurs enregistrements de données environnementales.
- **Indices (1,n) – (1,1) DataIndices** : Chaque type d'indice (précipitation, température, NDVI, etc.) peut correspondre à plusieurs données.
- **User (1,n) – (1,1) Role** : Chaque utilisateur possède exactement un rôle (USER, MODERATOR, ADMIN).
- **Province (1,n) – (1,1) DataIndices (bidirectionnelle)** : Cette relation bidirectionnelle permet de naviguer à la fois depuis la province vers les données et inversement.

Ce modèle garantit l'intégrité référentielle, facilite l'organisation des données et optimise les requêtes, en particulier les recherches spatiales et l'exploitation des indices environnementaux.

3.2.2 Dictionnaire de données

3.2.3 Modèle relationnel de données (MRD)

Le MRD définit les tables et contraintes :

- - PROVINCES (id PRIMARY KEY, name UNIQUE, longitude, latitude)
- - DATAINDICES (id PRIMARY KEY, indicator, country, adm1Code, province, landType, date, data, dataLongTermAverage, year, month, decade, unit, source, indices_id FOREIGN KEY, provinces_id FOREIGN KEY)
- - INDICES (id PRIMARY KEY, name)
- - USERS (id PRIMARY KEY, username, email, password, roles)
- - ROLES (id PRIMARY KEY, name)
- - FILES (id PRIMARY KEY, name, url, type, size)

TABLE 3.1 – Dictionnaire des données principales

Entité	Attribut	Type	Description
Province	id	BIGINT	Identifiant unique de la province
	name	VARCHAR	Nom de la province
	longitude	DOUBLE	Coordonnée géographique longitude
	latitude	DOUBLE	Coordonnée géographique latitude
DataIndices	id	BIGINT	Identifiant unique de l'enregistrement
	indicator	VARCHAR	Type d'indicateur (ex. : précipitations, NDVI)
	country	VARCHAR	Pays concerné
	adm1Code	BIGINT	Code administratif niveau 1
	province	VARCHAR	Nom de la province associée
	landType	VARCHAR	Type de terre (ex. : agricole, désertique)
	date	DATE	Date de l'enregistrement
	data	DOUBLE	Valeur mesurée de l'indicateur
	dataLongTermAverage	DOUBLE	Moyenne à long terme de l'indicateur
	year	BIGINT	Année de l'enregistrement
	month	BIGINT	Mois de l'enregistrement
	decade	BIGINT	Décennie de l'enregistrement
	unit	VARCHAR	Unité de mesure (ex. mm, °C)
	source	VARCHAR	Source de la donnée

3.3 Vue dynamique du système

La vue dynamique décrit les interactions temporelles

3.4 Déploiement du système

L'architecture de déploiement adoptée suit une approche **microservices-like** avec une séparation claire des composants :

- **Backend** : application Spring Boot déployée soit sur un serveur Tomcat, soit en JAR exécutable autonome.
- **Base de données** : H2 ou PostgreSQL, exécutée dans une instance dédiée.
- **Frontend** : application Angular servie via un serveur web (Nginx ou Apache).
- **Chatbot** : intégré soit via une API externe, soit comme service local.
- **Conteneurisation et orchestration** : utilisation de Docker pour faciliter le déploiement, la scalabilité et la résilience.

Cette architecture assure **la scalabilité horizontale**, la **résilience du système** et facilite la maintenance ainsi que les mises à jour indépendantes des composants. sure la scalabilité horizontale et la résilience du système.

3.5 Conclusion

L'étude conceptuelle a établi les fondations solides du système Plateforme Geo API, définissant sa structure statique et dynamique, ainsi que son modèle de données. Cette conception modulaire facilite la maintenance et l'évolution future du système, préparant efficacement la phase de réalisation.

Chapitre 4

Réalisation

4.1 Introduction

La phase de réalisation constitue l'implémentation concrète des spécifications et de l'étude conceptuelle établies précédemment. Cette étape a transformé les modèles théoriques en un système fonctionnel, en appliquant les meilleures pratiques de développement logiciel. Le développement a été réalisé de manière itérative, avec des tests continus pour assurer la qualité et la conformité aux exigences.

4.2 Environnement de travail

4.2.1 Environnement matériel

Le développement de l'application a été réalisé sur un ordinateur portable standard, présentant les caractéristiques suivantes :

- **Processeur** : Intel Core i5 (8ème génération)
- **Mémoire RAM** : 8 Go
- **Stockage** : Disque SSD de 256 Go
- **Système d'exploitation** : Windows 11

Cet environnement matériel a permis un développement fluide et efficace, avec la possibilité d'exécuter simultanément le backend Spring Boot, la base de données H2

4.2.2 Environnement logiciel

Le développement du système a été réalisé en utilisant un ensemble de technologies et d'outils modernes, adaptés au développement backend, frontend et à la gestion de la base de données. Les principaux éléments sont les suivants :

- **Java 11** : langage de programmation principal utilisé pour le développement du backend.
- **Spring Boot 2.6.6** : framework Java facilitant le développement rapide d'applications robustes et modulaires.
- **Angular 12** : framework frontend permettant la création d'interfaces utilisateur réactives et dynamiques.
- **H2 Database** : base de données embarquée utilisée pour le développement et les tests locaux.

- **PostgreSQL** : base de données relationnelle utilisée en environnement de production (optionnelle selon le déploiement).
- **Node.js 16** : environnement d'exécution JavaScript nécessaire pour Angular et la gestion des dépendances frontend.
- **Maven** : outil de gestion des dépendances et de compilation pour le projet Java.
- **Visual Studio Code** : éditeur de code principal utilisé pour le développement et le débogage.

Cette combinaison d'outils et de technologies a permis un développement efficace, structuré et modulable, facilitant la maintenance et l'évolution du système.

4.3 Architecture de l'application

L'architecture de l'application suit le modèle **MVC (Modèle-Vue-Contrôleur)**, avec une séparation claire des différentes couches fonctionnelles :

- **Couche présentation (Frontend)** : Angular gère l'interface utilisateur, les composants et la logique de présentation, offrant une expérience réactive et dynamique aux utilisateurs.
- **Couche métier (Backend)** : Spring Boot implémente la logique applicative à travers les services, assurant le traitement des données et la coordination des fonctionnalités.
- **Couche persistance** : JPA/Hibernate assure le mapping objet-relationnel entre les entités de l'application et la base de données, facilitant l'accès et la gestion des données.
- **Couche sécurité** : Spring Security gère l'authentification et l'autorisation, garantissant que seuls les utilisateurs autorisés peuvent accéder aux ressources du système.

La communication entre le frontend et le backend se fait via une **API REST**, sécurisée par un système d'authentification **JWT (JSON Web Token)**, garantissant la confidentialité et l'intégrité des échanges.

4.4 Présentation des interfaces

L'interface utilisateur a été conçue pour être à la fois **intuitive** et **responsive**, offrant une expérience fluide sur tous les types d'appareils. Les principales interfaces sont les suivantes :

- **Page de connexion/inscription** : formulaire d'authentification avec validation des champs en temps réel.

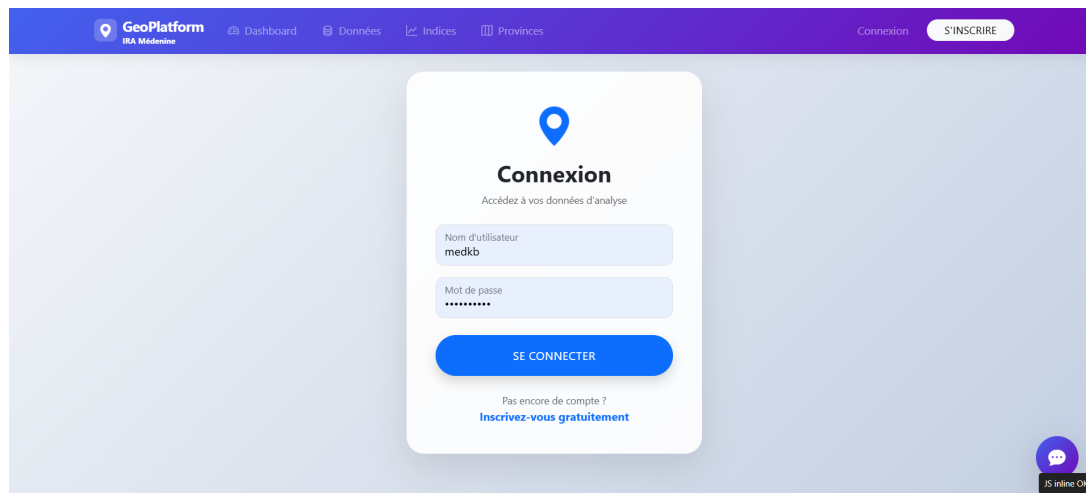


FIGURE 4.1 – Page de connexion

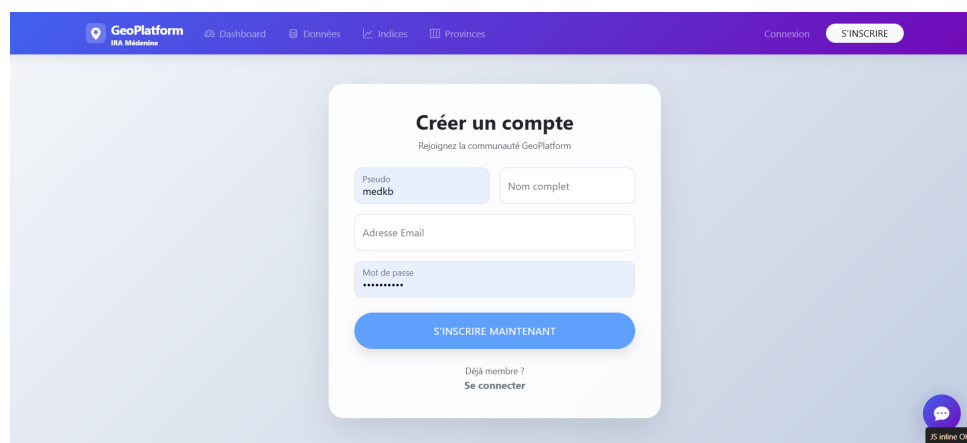


FIGURE 4.2 – Page de inscription

- **Tableau de bord** : vue d'ensemble présentant les statistiques clés et offrant un accès rapide aux différentes fonctionnalités de la plateforme.

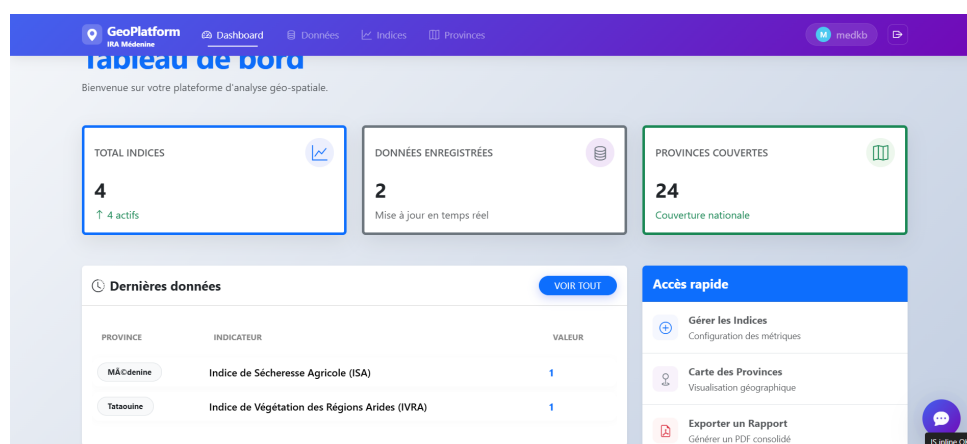


FIGURE 4.3 – Page de Tableau de bord

- **Données** : interface permet la consultation centralisée des indices géographiques par province et par année, tout en donnant accès aux publications scientifiques et projets de recherche associés.

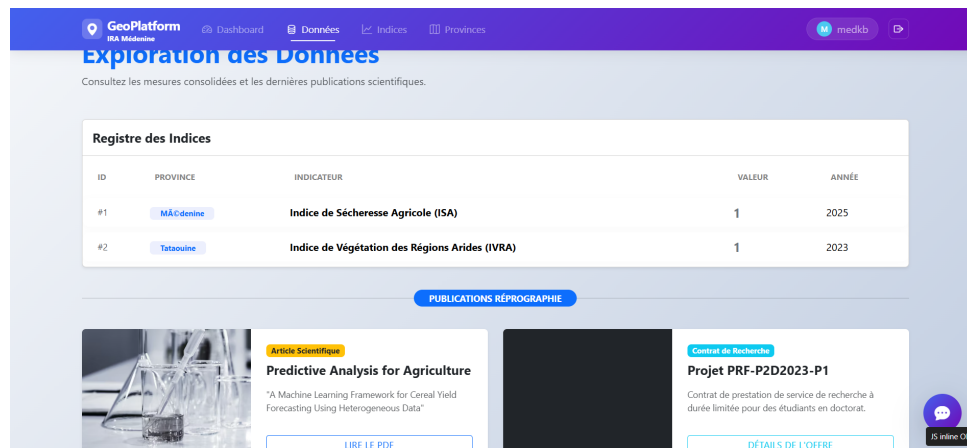


FIGURE 4.4 – Page de données

- **Gestion des indices** : tableaux interactifs pour la visualisation, l'édition et la mise à jour des données environnementales

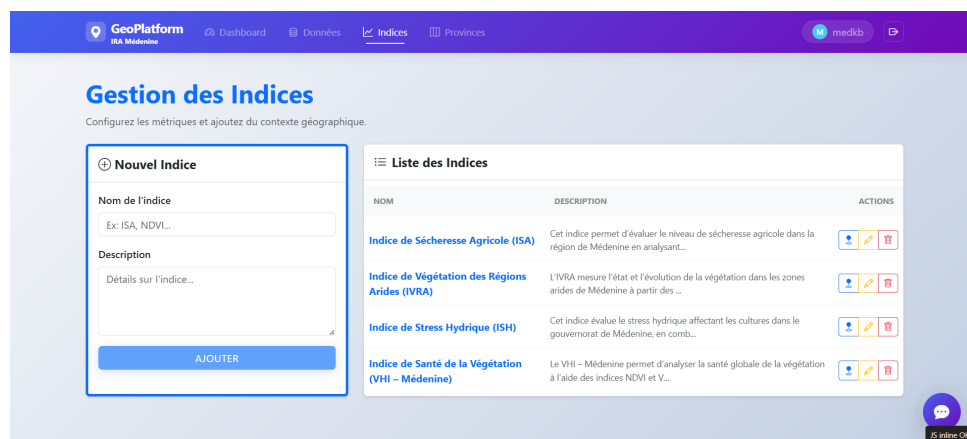


FIGURE 4.5 – Page de indices

- **Gestion des provinces** : interface CRUD permettant aux administrateurs de gérer les données géographiques.

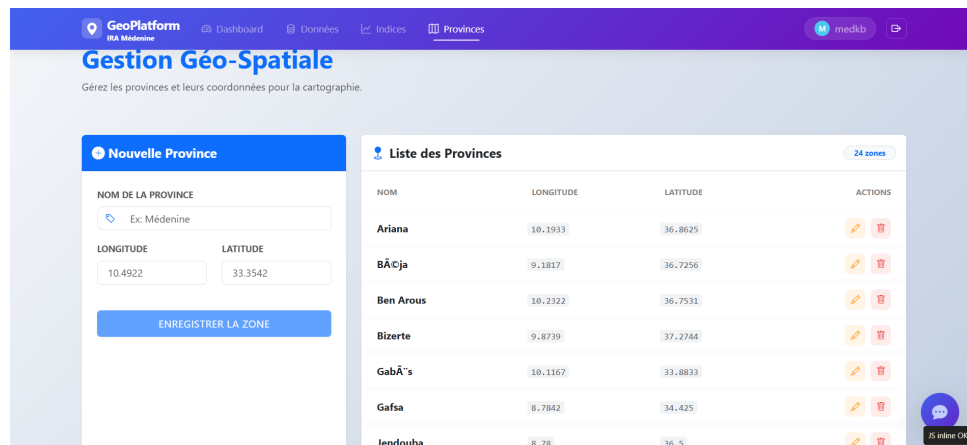


FIGURE 4.6 – Page de provinces

- **Chatbot** : interface de discussion fournissant une assistance interactive aux utilisateurs.



FIGURE 4.7 – Chatbot

- **Carte interactive** : intégration de Leaflet pour la visualisation cartographique des provinces et des données associées.

FIGURE 4.8 – Carte interactive

- **Upload de fichiers** : zone de dépôt permettant l'importation de fichiers CSV pour alimenter la base de données.

FIGURE 4.9 – Upload de fichiers

Toutes les interfaces respectent les principes d'ergonomie web, assurant une navigation intuitive et une compatibilité optimale avec les appareils mobiles.

4.5 Intégration de la base de données

L'intégration de la base de données a été réalisée à l'aide de ****JPA/Hibernate****, permettant un mapping objet-relationnel efficace et une gestion simplifiée des opérations CRUD. Les principales étapes sont les suivantes :

- Configuration des paramètres de connexion dans le fichier `application.properties`.
- Utilisation des annotations JPA sur les entités pour un mapping automatique entre les classes Java et les tables de la base de données.
- Emploi des *repositories* Spring Data pour simplifier les opérations CRUD et les requêtes.
- Gestion des transactions afin de garantir l'intégrité et la cohérence des données.
- Migration automatique du schéma de la base de données via Hibernate.

La base de données **H2** permet un démarrage rapide et pratique en environnement de développement, tandis que **PostgreSQL** est privilégiée en production pour ses meilleures performances, sa robustesse et sa gestion des données à grande échelle.

4.6 Intégration de la partie IA (Chatbot)

Le chatbot a été intégré comme un service externe : - Interface REST pour la communication avec le frontend - Traitement du langage naturel pour comprendre les requêtes utilisateur - Base de connaissances alimentée par les données de la plateforme - Réponses contextuelles basées sur les données géographiques et environnementales - Apprentissage continu pour améliorer la pertinence des réponses

Cette intégration enrichit l'expérience utilisateur en fournissant une assistance interactive et personnalisée.

4.7 Conclusion

La phase de réalisation a abouti à une plateforme fonctionnelle et robuste, respectant toutes les spécifications établies. Les choix technologiques ont permis de développer une application scalable et maintenable. Les tests unitaires et d'intégration ont validé le bon fonctionnement du système, préparant le terrain pour un déploiement en production.

Conclusion générale

Ce projet de développement de la Plateforme Geo API a été une expérience enrichissante qui a permis d'appliquer les concepts théoriques de l'ingénierie logicielle à un cas concret dans le domaine géospatial. Les objectifs principaux ont été atteints : création d'une plateforme unifiée pour la gestion et la visualisation des données géographiques marocaines, intégration d'un système d'authentification sécurisé, et mise en place d'une interface utilisateur intuitive avec un chatbot assisté par IA.

La plateforme offre désormais une gestion efficace des provinces et des indices environnementaux, facilitant l'accès aux données pour les chercheurs, décideurs et citoyens. L'architecture moderne adoptée assure une scalabilité et une maintenabilité élevées, permettant des évolutions futures.

Les défis rencontrés, tels que l'intégration des données géospatiales et la synchronisation frontend/backend, ont été relevés avec succès grâce à une approche itérative et des tests rigoureux. Le projet démontre l'importance des méthodologies de développement structurées dans la réalisation de systèmes complexes.

Perspectives d'amélioration incluent l'intégration de sources de données externes (APIs météorologiques, satellites), l'optimisation des performances pour de gros volumes de données, et l'extension du chatbot avec des capacités d'analyse prédictive. Ce projet ouvre la voie à des applications similaires dans d'autres domaines géospatiaux.

Au cours de ce travail, plusieurs sites web ont été consultés afin de collecter des informations, d’approfondir les connaissances techniques et d’installer les outils nécessaires au développement. Les principaux sites visités sont les suivants :

- **www.ira.agrinet.tn** : site officiel de l’Institut des Régions Arides (IRA), utilisé pour la consultation des informations institutionnelles.
- **<https://isetta.rnu.tn>** : plateforme universitaire dédiée à la gestion académique et administrative.
- **<https://www.linkedin.com>** : réseau professionnel utilisé pour la veille technologique et la consultation de profils et d’articles techniques.
- **www.hostinger.fr** : plateforme d’hébergement web utilisée pour se documenter sur les solutions de déploiement.
- **<https://stackoverflow.com>** : forum de référence pour la résolution de problèmes techniques liés au développement.
- **<https://openclassrooms.com>** : plateforme de formation en ligne utilisée pour renforcer les compétences en développement logiciel.

Annexes

4.8 Annexe A : Code source principal

Le code source complet de l'application est disponible dans le répertoire du projet. Les principaux fichiers et classes utilisés sont les suivants :

- `PlateformeGeoApiApplication.java` : classe principale de démarrage de l'application Spring Boot.
- `Province.java` : entité représentant les provinces avec leurs attributs géographiques.
- `DataIndices.java` : entité contenant les données environnementales et les indices associés.
- `AuthController.java` : contrôleur chargé de l'authentification et de la gestion des accès.
- `app.component.ts` : composant principal de l'application Angular.
-

4.9 Annexe B : Schémas et diagrammes détaillés

Cette annexe regroupe les différents schémas et diagrammes réalisés pour la modélisation du système :

- Diagramme de classes UML.
- Modèle conceptuel de données (MCD) détaillé.
- Diagrammes de séquence complets (authentification, consultation, import des données).
- Schéma de l'architecture de déploiement.

4.10 Annexe C : Fichiers de configuration

Les fichiers de configuration utilisés pour le bon fonctionnement de l'application sont les suivants :

- `application.properties` : fichier de configuration du backend Spring Boot.
- `proxy.conf.json` : fichier de configuration du proxy pour l'application Angular.
- `data.sql` : script SQL d'initialisation de la base de données.
-

4.11 Annexe D : Tests et validation

Cette annexe présente les différents types de tests réalisés afin de garantir la qualité et la fiabilité de l'application :

- Exemples de tests unitaires réalisés avec JUnit.
- Tests d'intégration pour la validation des endpoints REST.
- Tests end-to-end (E2E) avec Protractor, lorsque ceux-ci ont été implémentés.